🖥️ OS AUTOMATION — QUICK REVISION NOTES

Beginner → Advanced → Professional Level

Python | Exam + Interview + Real-World Development

Core idea: OS Automation means using Python to interact with the operating system automatically—files, folders, paths, processes, commands, backups, cleanup, and repetitive computer tasks.

🗺️ COMPLETE SYLLABUS

Level	Level Name	Main Topics
Level 1	🌱 Beginner Level	`os`, working directory, files/folders, basic operations
Level 2	🟢 Intermediate Level	`pathlib`, paths, searching, file metadata
Level 3	🔵 Intermediate+ Level	`shutil`, copy, move, rename, directory trees
Level 4	🟠 Advanced Level	`glob`, recursive search, pattern matching, `subprocess`
Level 5	🔴 Professional/Expert Level	safety, errors, logging, validation, dry-run, configuration, OOP, testing

---

LEVEL 1 — 🌱 BEGINNER LEVEL

OS + `os` Module

---

1. What is OS Automation?

Normally:

```
User
 ↓
Manually performs task
 ↓
Computer
```

Automation:

```
Python program
 ↓
Operating System
 ↓
Computer performs task
```

Examples

```
Create folders
Rename files
Move files
Copy files
Delete files
Find files
Backup files
Run commands
Start programs
Clean directories
Organize Downloads
```

---

2. Import `os`

```
import os
```

`os` provides Python's interface to operating-system functionality.

---

3. Current Working Directory

```
import os

print(os.getcwd())
```

`getcwd()`

Get Current Working Directory

It tells you where Python is currently working.

```
C:\Users\User\Projects
```

or:

```
/home/user/projects
```

Important

```
open("data.txt")
```

looks for `data.txt` relative to the current working directory.

---

4. List Directory Contents

```
os.listdir()
```

Example:

```
import os

print(os.listdir())
```

Possible output:

```
['app.py', 'data.txt', 'Images', 'Documents']
```

Specific directory:

```
os.listdir("Documents")
```

---

5. File vs Directory

```
os.path.isfile(path)
os.path.isdir(path)
```

Example:

```
for item in os.listdir():

    if os.path.isfile(item):
        print("FILE:", item)

    elif os.path.isdir(item):
        print("FOLDER:", item)
```

Function	Checks
`isfile()`	Is it a file?
`isdir()`	Is it a directory?

---

6. Check Existence

```
os.path.exists("data.txt")
```

Returns:

```
True
```

or:

```
False
```

Professional pattern:

```
if os.path.exists("data.txt"):
    print("Exists")
else:
    print("Not found")
```

---

7. Create Directory

One directory

```
os.mkdir("Images")
```

Nested directories

```
os.makedirs("Data/Images/2026")
```

Don't fail if already exists

```
os.makedirs("Data/Images", exist_ok=True)
```

Method	Purpose
`mkdir()`	Create one directory
`makedirs()`	Create nested directories
`exist_ok=True`	Don't raise error if already exists

---

8. Relative vs Absolute Path

Relative

```
data/report.pdf
```

Means:

Find `data/report.pdf` relative to current location.

Absolute

Windows:

```
C:\Users\User\Documents\report.pdf
```

Linux/macOS:

```
/home/user/Documents/report.pdf
```

Remember

```
Relative → from where I am
Absolute → complete location
```

---

9. `os.path.join()`

❌ Avoid manually constructing paths:

```
path = "data/" + "images/" + "photo.jpg"
```

✅ Prefer:

```
path = os.path.join(
    "data",
    "images",
    "photo.jpg"
)
```

It creates a platform-appropriate path.

---

10. `os.path.abspath()`

Convert relative path → absolute path.

```
path = os.path.abspath("data.txt")

print(path)
```

---

11. `basename()`

```
path = "/home/user/Documents/report.pdf"

print(os.path.basename(path))
```

Output:

```
report.pdf
```

Meaning

```
Full path
   ↓
basename
   ↓
last component
```

---

12. `dirname()`

```
print(os.path.dirname(path))
```

Output:

```
/home/user/Documents
```

Function	Result
`basename()`	`report.pdf`
`dirname()`	parent directory

---

13. `splitext()`

```
name, extension = os.path.splitext("report.pdf")

print(name)
print(extension)
```

Output:

```
report
.pdf
```

Useful for file classification.

```
if extension.lower() == ".pdf":
    print("Document")
```

---

14. Rename

```
os.rename("old.txt", "new.txt")
```

Works for files and directories.

---

15. Delete File

```
os.remove("test.txt")
```

⚠️ Destructive operation.

---

16. Delete Empty Directory

```
os.rmdir("EmptyFolder")
```

Important:

The directory must be empty.

---

17. `os.walk()`

Used for recursive directory traversal.

```
for root, dirs, files in os.walk("Project"):
    print(root)
    print(dirs)
    print(files)
```

Think:

Variable	Meaning
`root`	Current directory
`dirs`	Directories inside it
`files`	Files inside it

Find All PDFs

```
for root, dirs, files in os.walk("Project"):

    for file in files:

        if file.lower().endswith(".pdf"):
            print(os.path.join(root, file))
```

`os.walk()` is useful for

```
File search
Backup
Cleanup
Duplicate detection
Indexing
Log analysis
File organizers
```

LEVEL 2 — 🟢 INTERMEDIATE LEVEL

`pathlib`

---

18. Why `pathlib`?

Modern Python code often prefers `pathlib` for path manipulation.

```
from pathlib import Path
```

Compare:

```
os.path.join("data", "images", "photo.jpg")
```

with:

```
Path("data") / "images" / "photo.jpg"
```

The second is often easier to read.

---

19. Create a `Path`

```
from pathlib import Path

path = Path("data")
```

A `Path` object represents a filesystem path.

---

20. Current Directory

`Path.cwd()`

Equivalent idea:

`os.getcwd()`

---

21. Home Directory

`Path.home()`

Returns the current user's home directory.

---

22. Joining Paths

```
from pathlib import Path

path = Path("data") / "images" / "photo.jpg"
```

Important:

```
Path("data") / "images"
```

does **path joining**, not mathematical division.

This is also a nice example of operator overloading from OOP.

---

23. Existence

`path.exists()`

---

24. File / Directory

`path.is_file()`
`path.is_dir()`

Example:

```
if path.is_file():
    print("File")

elif path.is_dir():
    print("Directory")
```

---

25. Create Directory

```
Path("Images").mkdir()
```

Nested:

```
Path("Data/Images/2026").mkdir(
    parents=True,
    exist_ok=True
)
```

---

26. `iterdir()`

Equivalent idea to `os.listdir()` but returns `Path` objects.

```
folder = Path(".")

for item in folder.iterdir():
    print(item)
```

Professional-style scan:

```
for item in folder.iterdir():

    if item.is_file():
        print("FILE:", item)

    elif item.is_dir():
        print("DIR:", item)
```

---

27. Important `Path` Properties

Given:

```
path = Path("documents/report.pdf")
```

Expression	Result / Meaning
`path.name`	`report.pdf`
`path.stem`	`report`
`path.suffix`	`.pdf`
`path.parent`	`documents`
`path.parts`	Path components
`path.exists()`	Existence
`path.is_file()`	File check
`path.is_dir()`	Directory check

---

28. `resolve()`

```
path.resolve()
```

Used to obtain a resolved/absolute form of a path.

Example:

```
print(Path("data.txt").resolve())
```

---

29. Rename

```
file = Path("old.txt")

file.rename("new.txt")
```

---

30. Delete File

```
file.unlink()
```

Roughly analogous to:

```
os.remove(...)
```

---

31. Delete Empty Directory

```
folder.rmdir()
```

Again:

Directory must be empty.

---

32. `glob()`

Search files matching a pattern.

```
folder = Path("Project")

for file in folder.glob("*.pdf"):
    print(file)
```

Finds PDFs in the current directory.

---

33. `rglob()`

Recursive search:

```
for file in folder.rglob("*.pdf"):
    print(file)
```

Method	Search
`glob()`	Current directory/pattern scope

`rglob()`	Recursive subdirectories

---

34. File Metadata

```
info = file.stat()
```

Useful attributes include:

```
file.stat().st_size
```

which gives file size in bytes.

Convert to MB:

```
size_mb = file.stat().st_size / (1024 * 1024)
```

LEVEL 3 — 🔵 INTERMEDIATE+ LEVEL

`shutil`

---

35. What is `shutil`?

```
import shutil
```

Provides higher-level file and directory operations.

Main operations:

```
copy
move
directory-tree copying
directory-tree deletion
```

---

36. Copy a File

```
shutil.copy(
    "source.txt",
    "backup.txt"
)
```

---

37. Copy to Directory

```
shutil.copy(
    "report.pdf",
    "Documents/"
)
```

---

38. `copy()` VS `copy2()`

`shutil.copy()`

Copies the file.

`shutil.copy2()`

Copies the file and attempts to preserve additional metadata.

---

39. Move

```
shutil.move(
    "report.pdf",
    "Documents/"
)
```

Unlike copying, the original location no longer contains the file.

---

40. Copy Directory

```
shutil.copytree(
    "source_folder",
    "backup_folder"
)
```

Copies the directory tree.

---

41. Delete Directory Tree

```
shutil.rmtree("folder")
```

⚠️ **Dangerous.**

It can recursively delete a directory and its contents.

Never experiment with this on important folders.

---

42. Safe File Operation Pattern

Professional automation:

```
Input
 ↓
Validate
 ↓
Preview
 ↓
Execute
 ↓
Log
 ↓
Verify
```

Not:

```
Input
 ↓
Immediately modify/delete
```

LEVEL 4 — 🔴 ADVANCED LEVEL

Search + `subprocess`

---

43. `glob` Module

```
import glob

files = glob.glob("*.pdf")
```

Possible result:

```
['report.pdf', 'notes.pdf']
```

---

44. Common Patterns

Pattern	Meaning
`*.pdf`	Any PDF
`*.txt`	Any TXT
`photo*`	Starts with `photo`
`file?.txt`	One-character wildcard
`**/*.pdf`	Recursive pattern

For modern path-oriented code, also know:

```
Path.glob()
Path.rglob()
```

---

45. Case-Insensitive Extension Handling

Instead of:

```
if file.suffix == ".pdf":
```

prefer:

```
if file.suffix.lower() == ".pdf":
```

Now these can all match:

```
.pdf
.PDF
.Pdf
.pDf
```

---

46. File Classification

```
from pathlib import Path

folder = Path("Downloads")

for file in folder.iterdir():

    if not file.is_file():
        continue

    extension = file.suffix.lower()

    if extension in [".jpg", ".jpeg", ".png"]:
        category = "Images"

    elif extension in [".pdf", ".docx", ".txt"]:
```

```
        category = "Documents"

    elif extension in [".mp4", ".mkv"]:
        category = "Videos"

    elif extension in [".mp3", ".wav"]:
        category = "Audio"

    else:
        category = "Other"

    print(file.name, "→", category)
```

Important professional concept

Separate:

```
Classification
```

from:

```
Action
```

Don't immediately move files while deciding their category.

---

47. `subprocess`

Used to run external programs/commands.

```
import subprocess
```

Example:

```
result = subprocess.run(
    ["python", "--version"],
    capture_output=True,
    text=True
)

print(result.stdout)
```

---

48. `capture_output=True`

Captures:

```
stdout
stderr
```

instead of simply allowing output to go directly to the console.

---

49. `text=True`

Makes command output available as normal Python strings rather than bytes.

---

50. Return Code

```
print(result.returncode)
```

Generally:

```
0       → success
non-zero → failure
```

Example:

```
if result.returncode == 0:
    print("Success")
else:
    print("Failed")
```

---

51. `check=True`

```
subprocess.run(
    ["some_program"],
    check=True
)
```

If the command fails, Python raises an exception.

Useful when command failure should be treated as an error.

---

52. `shell=True`

You may see:

```
subprocess.run(
    "some command",
    shell=True
)
```

⚠️ Don't use it casually.

If untrusted user input is inserted into a shell command, it can create serious security vulnerabilities.

Prefer:

```
subprocess.run(
    ["program", "argument"]
)
```

when possible.

LEVEL 5 — 🔴 PROFESSIONAL / EXPERT LEVEL

Safe, Maintainable Automation

---

53. Error Handling

OS operations can fail.

Possible reasons:

```
File doesn't exist
Permission denied
Invalid path
Directory not empty
File unavailable
External command failed
```

Use specific exceptions.

```
try:
    file.unlink()

except FileNotFoundError:
    print("File not found.")

except PermissionError:
    print("Permission denied.")
```

---

54. Avoid Blind `except`

❌ Bad:

```
try:
    ...
except:
    pass
```

Problems:

```
Hides errors
Makes debugging difficult
Can silently corrupt automation results
```

✅ Prefer specific exceptions:

```
except FileNotFoundError:
except PermissionError:
except OSError:
```

when appropriate.

---

55. Logging

Professional automation uses:

```
import logging
```

Example:

```
logging.basicConfig(
    level=logging.INFO
)

logging.info("Starting organizer")
logging.warning("Unknown extension")
logging.error("Move failed")
```

Log levels

Level	Use
`DEBUG`	Detailed debugging
`INFO`	Normal operation
`WARNING`	Something unusual
`ERROR`	Operation failed
`CRITICAL`	Serious failure

---

56. Why Logging?

Instead of:

```
Done!
```

you can have:

```
INFO   Starting organizer
INFO   Found 125 files
INFO   Moved report.pdf
WARNING Unknown extension:  .xyz
ERROR Permission denied
```

Useful for:

```
Debugging
Auditing
Production automation
Troubleshooting
Monitoring
```

---

57. Dry Run

A dry run means:

Show what the program **would do**, without actually changing the filesystem.

Example:

```
DRY RUN

photo.jpg  → Images/photo.jpg
report.pdf → Documents/report.pdf
movie.mp4  → Videos/movie.mp4
```

Nothing is moved.

Why?

Because:

```
Automation
 ↓
Potentially thousands of changes
```

Dry run allows:

```
Plan
 ↓
Review
 ↓
Fix
 ↓
```

```
Execute
```

---

58. Validation

Before performing operations:

```
if not source.exists():
    raise FileNotFoundError(
        "Source does not exist"
    )
```

Professional automation validates:

```
Source
Destination
Permissions
File type
Arguments
Configuration
Operation
```

---

59. Collision Handling

Suppose:

```
Downloads/report.pdf
```

must move to:

```
Documents/report.pdf
```

but that file already exists.

Possible policies:

Strategy	Behaviour
Skip	Don't move
Overwrite	Replace existing file
Rename	`report_1.pdf`
Ask	User decides
Version	Keep multiple copies

Never leave collision behaviour undefined in a serious automation system.

---

60. Permissions

A file existing does **not** guarantee you can modify it.

Possible:

```
PermissionError
```

Therefore:

```
exists ≠ writable
```

Your program must handle permissions appropriately.

---

61. File Locks

Another application may be using a file:

```
Excel
Word
Database
Video editor
```

Therefore:

```
File exists
     ≠
File is currently available for modification
```

Your operation still needs error handling.

---

62. Race Conditions

Example:

```
Python checks:
"file exists"
       ↓
Another program deletes it
       ↓
Python tries to move it
       ↓
Operation fails
```

Therefore, this:

```
if path.exists():
    path.unlink()
```

doesn't guarantee the operation will succeed.

Always handle the actual filesystem operation.

---

63. Configuration

Don't hard-code every setting:

```
folder = "C:/Users/User/Downloads"
```

A professional program can get settings from:

```
JSON
Environment variables
Command-line arguments
Configuration files
```

Possible settings:

```
Source folder
Destination folder
File extensions
Dry-run
Collision policy
Logging level
```

---

64. OOP + OS AUTOMATION

Your OOP knowledge becomes extremely useful here.

Instead of one huge script:

```
1000-line file organizer
```

design separate components:

```
FileScanner
FileClassifier
OperationPlanner
FileMover
Logger
FileOrganizer
```

Example:

```
class FileScanner:

    def scan(self, folder):
        ...
```

```
class FileClassifier:

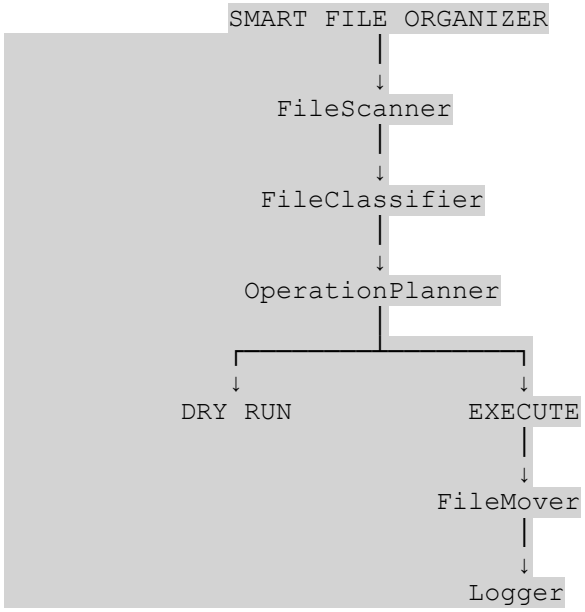
    def classify(self, file):
        ...
class FileMover:

    def move(self, source, destination):
        ...
```

This produces maintainable automation.

---

65. Professional Architecture



Underlying tools:

```
pathlib
shutil
os
subprocess
exceptions
logging
OOP
testing
```

---

66. Real-World OS Automation Applications

Application	Main Tools
Download organizer	`pathlib`, `shutil`

Backup system	`shutil`, `pathlib`
Log cleanup	`Path`, `glob`
File search	`rglob`, `os.walk`
Bulk rename	`Path.rename()`
Duplicate analysis	`Path`, hashing
Disk analyzer	`Path.stat()`
Program launcher	`subprocess`
Automated deployment	`subprocess`, `os`
Report generator	`pathlib`, `logging`
Temporary-file cleanup	`pathlib`
Batch processing	`pathlib`, `subprocess`

🧠 MOST IMPORTANT COMPARISONS

os VS pathlib

os	pathlib
`os.getcwd()`	`Path.cwd()`
`os.listdir()`	`Path.iterdir()`
`os.path.exists()`	`Path.exists()`
`os.path.isfile()`	`Path.is_file()`
`os.path.isdir()`	`Path.is_dir()`
`os.mkdir()`	`Path.mkdir()`
`os.rename()`	`Path.rename()`
`os.remove()`	`Path.unlink()`
`os.rmdir()`	`Path.rmdir()`
`os.path.join()`	`Path / "child"`
`os.path.splitext()`	`Path.suffix` / `Path.stem`

Rule of thumb

Know **both**, but prefer `pathlib` for modern path-oriented Python code.

---

os VS shutil

os	shutil
OS interface	High-level file operations
Paths	Copy
Basic filesystem operations	Move
Environment	Directory trees
`os.remove()`	`copy()`
`os.rename()`	`move()`
`os.walk()`	`copytree()`
`os.rmdir()`	`rmtree()`

`glob()` VS `rglob()`

```
folder.glob("*.pdf")
```

→ search according to pattern at that directory level.

```
folder.rglob("*.pdf")
```

→ recursive search through subdirectories.

`copy()` VS `move()`

```
shutil.copy(source, destination)
```

→ original remains.

```
shutil.move(source, destination)
```

→ original is moved from its previous location.

`remove()` VS `rmdir()` VS `rmtree()`

Operation	Used for
`os.remove()`	File
`Path.unlink()`	File
`os.rmdir()`	Empty directory
`Path.rmdir()`	Empty directory
`shutil.rmtree()`	Directory tree

🏆 PROFESSIONAL FILE ORGANIZER — FINAL CONCEPT

A professional implementation should follow:

```
USER INPUT
    ↓
VALIDATION
```

```
                ↓
          FILE SCANNING
                ↓
         CLASSIFICATION
                ↓
          OPERATION PLAN
                ↓
            DRY RUN
                ↓
        USER/CONFIG CHECK
                ↓
           EXECUTION
                ↓
          ERROR HANDLING
                ↓
             LOGGING
                ↓
           VERIFICATION
```

This architecture is far more important than simply memorizing functions.

🎯 INTERVIEW + EXAM QUESTIONS

Most Expected Questions

Beginner

1. What is OS automation in Python?

2. What is the difference between a file and a directory?

3. What does `os.getcwd()` do?

4. What does `os.listdir()` return?

5. Difference between `os.mkdir()` and `os.makedirs()`?

6. Difference between relative and absolute paths?

7. Why should `os.path.join()` be used instead of manually concatenating paths?

Intermediate

8. What is `pathlib` and why is it preferred for path handling?

9. Difference between `Path.name`, `Path.stem`, and `Path.suffix`?

10. Difference between `Path.glob()` and `Path.rglob()`?

11. How do you check whether a path is a file or directory?

12. What is `Path.iterdir()`?

13. What does `Path.stat()` provide?

Intermediate+

14. Difference between `shutil.copy()` and `shutil.copy2()`?

15. Difference between copying and moving a file?

16. What does `shutil.copytree()` do?

17. Why is `shutil.rmtree()` dangerous?

18. How would you recursively find all PDF files?

Advanced

19. What is `subprocess.run()` used for?

20. What is the purpose of `capture_output=True`?

21. What does `text=True` do?

22. What is a subprocess return code?

23. What does `check=True` do?

24. Why can `shell=True` be dangerous?

25. How would you safely execute an external command with arguments?

Professional / Expert

26. Why should OS automation use specific exception handling?

27. Why is `except: pass` considered bad practice?

28. What is a dry-run mode and why is it useful?

29. How would you handle filename collisions during automation?

30. Why should classification and execution be separated?

31. How would you design a production-grade file organizer?

32. Where would OOP fit into an OS automation application?

33. How would you make a file automation script safe against accidental deletion?

34. Why does `exists()` not guarantee that a subsequent operation will succeed?

35. What is a race condition in filesystem automation?

36. Why is logging better than relying only on `print()` in production automation?

37. How would you test an automation script without risking real files?